

Project Phase I: Sequential RISC-V Processor Design

Prasanna N R (2024102056)

Pranavadarsh (2024102028)

Praveen V (2024102019)

Abstract—This project presents the design and implementation of a modular single-cycle RISC-V processor datapath using Verilog HDL. Core components including the program counter, instruction memory, control unit, register file, immediate generator, ALU, ALU control, and data memory are developed and verified independently before full integration. The processor successfully executes arithmetic, memory, and branch instructions, demonstrating correct instruction fetch, decode, and execution behavior. This work provides a structured foundation for extending the design toward pipelined and performance-optimized architectures.

I. INTRODUCTION

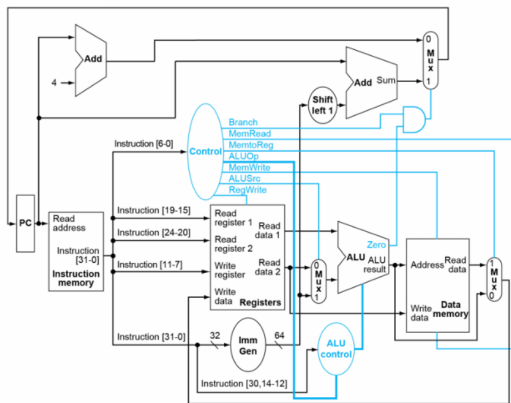


Fig. 1. Sequential datapath Diagram

The sequential processing model is the simplest form of instruction execution, where each instruction completes all stages of operation within a single clock cycle. This simplicity eliminates the need for complex hazard detection and forwarding mechanisms required in pipelined architectures. Additionally, the sequential approach enables a straightforward datapath design and makes functional verification and debugging significantly easier.

However, sequential execution comes with certain disadvantages:

- Low throughput
- Longer execution time
- Poor hardware utilization

II. PROJECT OBJECTIVES

The primary objective of this project is to design and implement a functional sequential RISC-V processor datapath

using modular Verilog HDL. The design emphasizes clarity, correctness, and adherence to the RISC-V base instruction formats while maintaining simplicity suitable for academic learning.

The specific goals of the project include:

- Understanding the internal datapath of a RISC-V processor
- Designing individual hardware blocks and integrating them systematically
- Verifying correct execution of arithmetic, memory, and branch instructions
- Gaining experience with hardware modeling using Verilog HDL

This implementation serves as a foundation for future extensions toward pipelined and high-performance processor architectures.

III. INSTRUCTIONS IMPLEMENTED

The processor implementation supports a subset of the RISC-V instruction set architecture (ISA), covering arithmetic, logical, memory access, and control flow operations. The supported instructions are categorized as follows:

R-type Instructions

- add
- sub
- and
- or

I-type Instructions

- addi
- ld

S-type Instruction

- sd

B-type Instruction

- beq

IV. CLOCKING METHODOLOGY

The designed processor follows a fully synchronous, edge-triggered clocking scheme. All state holding elements such as the program counter, register file, and data memory operate on the rising edge of the system clock. At each positive clock edge, the current instruction completes its execution and the processor state is updated simultaneously.

This edge-triggered approach ensures deterministic behavior, avoids race conditions between combinational blocks,

and simplifies timing analysis. Since the processor is single-cycle, the clock period is chosen to accommodate the worst case combinational delay across the datapath, ensuring correct operation for all supported instructions.

V. DATAPATH BLOCK DESCRIPTIONS

A. Program Counter (PC)

Program counter (PC) The register is a state element (i.e. 64-bit register) containing the address of the instruction in the program being executed and that is written at the end of every clock cycle.

Need: The program counter is required to hold the address of the next instruction to be fetched from instruction memory.

B. Register File

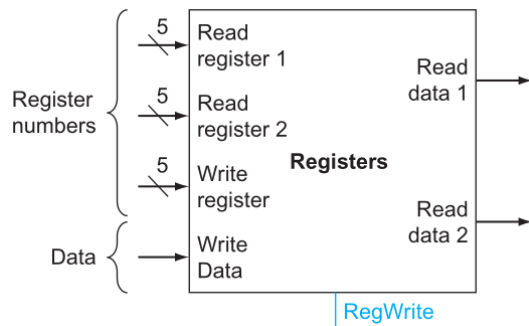


Fig. 2. Register Diagram

A state element that consists of a set of registers that can be read and written by supplying a register number to be accessed.

Need: The register file is required to provide fast storage and retrieval of operands during instruction execution.

C. Instruction Memory

Instruction memory is a state element that functions as a read only memory (ROM) block during program execution.

Need: The instruction memory is required to store the program instructions and provide the appropriate instruction to the processor based on the address supplied by the program counter during the fetch stage.

D. Control Unit

A control Unit is used to determine the values of control signal that are used in the different parts of the processor.

The outputs of the control unit consist of two 1-bit signals that are used to control multiplexors (ALUSrc and MemtoReg), three signals for controlling reads and writes in the register file and data memory (RegWrite, MemRead, and MemWrite), a 1-bit signal used in determining whether to possibly branch (Branch), and a 2-bit control signal for the

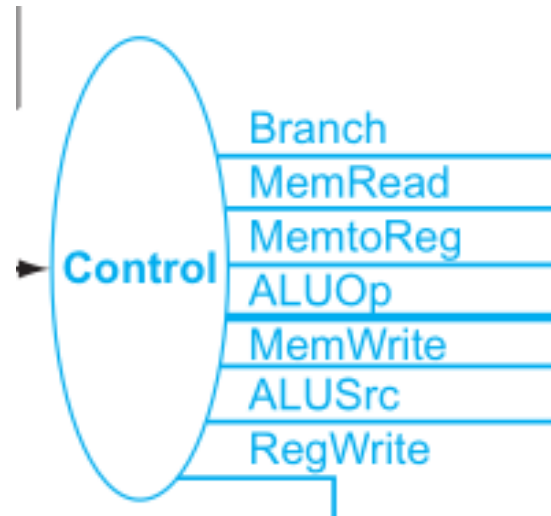


Fig. 3. Control unit Diagram

ALU (ALUOp). An AND gate is used to combine the branch control signal and the Zero output from the ALU; the AND gate output controls the selection of the next PC.

Need: The control unit is required to decode the fetched instruction and generate the appropriate control signals that coordinate the operation of the datapath components. It ensures that the correct data paths are enabled, the proper ALU operation is selected, and memory and register accesses occur in the required sequence for correct instruction execution.

E. Immediate Generation

The immediate generation unit (ImmGen) has a 32-bit instruction as input that selects a 12-bit field for load, store, and branch if equal that is sign-extended into a 64-bit result appearing on the output

F. ALU Control

We generate the 4-bit ALU control input using a small control unit that has as inputs the funct7 and funct3 fields of the instruction and a 2-bit control field, which we call ALUOp. ALUOp indicates whether the operation to be performed should be add (00) for loads and stores, subtract and test if zero (01) for beq, or be determined by the operation encoded in the funct7 and funct3 fields (10)

G. Arithmetic Logic Unit (ALU)

The arithmetic logic unit (ALU) is the brawn of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR.

H. Data Memory

Data memory is a state element that stores program data and provides read and write access during load and store instructions based on the address generated by the ALU.

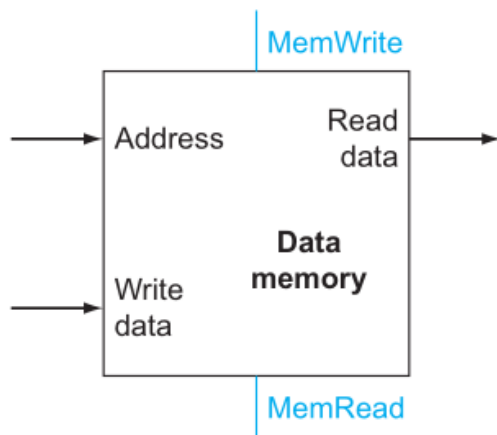


Fig. 4. data memory Diagram

I. Adder Blocks

The adder block is used to update the Pc by 4 after each instruction except in branch instructions where it is used to calculate the target branch address.

VI. INSTRUCTION SUBSET AND ENDIANNESS

The processor supports a compact subset of the RISC-V instruction set architecture sufficient to demonstrate arithmetic computation, memory access, and control flow. The implemented instructions include:

- Arithmetic: add, sub, addi
- Logical: and, or
- Memory access: ld, sd
- Branching: beq

This subset enables implementation of iterative programs, data manipulation, and conditional execution while maintaining a simple and verifiable datapath.

A. Big-Endian Memory Organization

Both the instruction memory (RegMem) and data memory (DataMem) are implemented using a Big-Endian format. In this approach, the most significant byte of a word is stored at the lowest memory address.

Although the standard RISC-V specification and textbook references typically use Little-Endian organization, a Big-Endian scheme is adopted in this design for simplicity of instruction representation and ease of debugging during simulation. All instruction fetch operations and load/store accesses follow this Big-Endian convention consistently across the processor.

This ensures uniform memory interpretation while maintaining functional correctness of the datapath.

VII. INSTRUCTION FLOW

A. R-type Instruction Flow

1. The instruction is fetched, and the PC is incremented.

2. Two registers, x2 and x3, are read from the register file; also, the main control unit computes the setting of the control lines during this step.

3. The ALU operates on the data read from the register file, using portions of the opcode to generate the ALU function.

4. The result from the ALU is written into the destination register (x1) in the register file.

B. Load (ld) Instruction Flow

1. An instruction is fetched from the instruction memory, and the PC is incremented.

2. A register (x2) value is read from the register file.

3. The ALU computes the sum of the value read from the register file and the sign-extended 12 bits of the instruction (offset).

4. The sum from the ALU is used as the address for the data memory.

5. The data from the memory unit is written into the register file (x1).

C. Immediate Arithmetic (addi) Instruction Flow

1. An instruction is fetched from the instruction memory, and the PC is incremented.

2. A register (x2) value is read from the register file.

3. The ALU computes the sum of the value read from the register file and the sign-extended 12 bits of the instruction (immediate).

4. The result from the ALU is written into the destination register (x1) in the register file.

D. Store (sd) Instruction Flow

1. An instruction is fetched from the instruction memory, and the PC is incremented.

2. Two register values (x2 as base address and x1 as data to be stored) are read from the register file.

3. The ALU computes the sum of the value read from the register file (x2) and the sign-extended 12 bits of the instruction (offset).

4. The sum from the ALU is used as the address for the data memory.

5. The data from the register file (x1) is written into the data memory at the computed address.

E. Branch (beq) Instruction Flow

1. An instruction is fetched from the instruction memory, and the PC is incremented.
2. Two registers, x1 and x2, are read from the register file.
3. The ALU subtracts one data value from the other data value, both read from the register file. The value of PC is added to the sign-extended, 12 bits of the instruction (offset) left shifted by one; the result is the branch target address.
4. The Zero status information from the ALU is used to decide which adder result to store in the PC.

VIII. DESIGN DETAILS OF VARIOUS STAGES

The processor follows a single-cycle datapath where each instruction completes in one clock cycle. For clarity, the operation is described in five logical stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

A. Instruction Fetch (IF)

In the Instruction Fetch stage, the Program Counter (PC) supplies the address of the next instruction to the instruction memory. The instruction memory outputs the 32-bit instruction corresponding to the current PC value.

An adder simultaneously computes $PC + 4$ to point to the next sequential instruction. A multiplexer controlled by the branch decision selects between the sequential address and the branch target address according to control value of Branch and the Zero output of the ALU. On every rising clock edge, the PC is updated with the selected next address.

B. Instruction Decode (ID)

During the decode stage, the fetched instruction fields are interpreted by the control unit to generate the required control signals. The register file reads the source operands specified by the instruction.

The immediate generator extracts and sign-extends the immediate field when required (for I-type, load, store, and branch instructions).

For the BEQ instruction, the immediate provided in the instruction is first stored in a right-shifted form for better storage, and during branch target computation it is shifted left by one bit to restore the correct byte offset before updating the program counter.

C. Execute Stage (EX)

In the Execute stage, the Arithmetic Logic Unit (ALU) performs arithmetic or logical operations. The first ALU operand comes from the register file, while the second operand is selected by the ALUSrc multiplexer between register data and the immediate value.

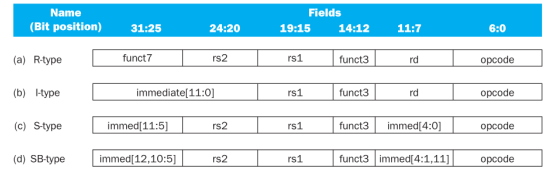


Fig. 5. instruction fields of instruction types Diagram

The ALU control unit determines the specific ALU operation using the ALUOp signal and instruction function fields. For branch instructions, the ALU also produces the Zero flag used for branch decisions.

D. Memory Access (MEM)

The Memory stage is active only for load and store instructions. The ALU result serves as the effective memory address.

- If MemRead is asserted, data memory is read (load instruction).
- If MemWrite is asserted, register data is written to memory (store instruction).

For non-memory instructions, this stage is bypassed.

E. Write Back (WB)

In the Write Back stage, the result is written into the destination register when RegWrite is enabled. The MemtoReg multiplexer selects the correct write-back source:

- ALU result for arithmetic and logical instructions
- Memory data for load instructions

IX. SIMULATION AND VERIFICATION

The designed processor was verified using Verilog testbenches covering all supported instruction types. Custom assembly programs were encoded into machine code and loaded into instruction memory to validate complete datapath functionality.

The following programs were successfully executed:

- Fibonacci sequence generation
- Linear search in an array
- Computation of the sum of first n natural numbers

These programs validate arithmetic operations, memory access, branching behavior, and iterative control flow. Simulation waveforms confirmed correct program counter updates, accurate register writes, proper ALU computations, and consistent memory read/write behavior.

Branch instructions were verified by observing correct control transfer based on ALU Zero output, ensuring reliable conditional execution. The processor demonstrated correct end-to-end execution for all tested scenarios.

A. Simulation and Verification

Since the complete simulation extends up to 1 μs , presenting the full waveform in a single view leads to significant signal congestion and reduced clarity. To ensure effective analysis, representative time intervals were selected and visualized.

Waveforms were captured at:

- 50 ns — Initial instruction execution
- 500 ns — Intermediate datapath operation
- Final time — Program completion state

These snapshots clearly demonstrate correct instruction fetch, ALU computation, branching behavior, and memory access while maintaining waveform readability.

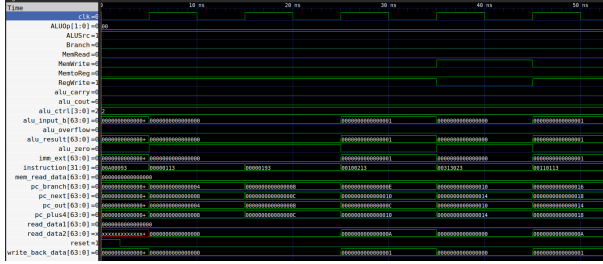


Fig. 6. Waveform at 50 ns

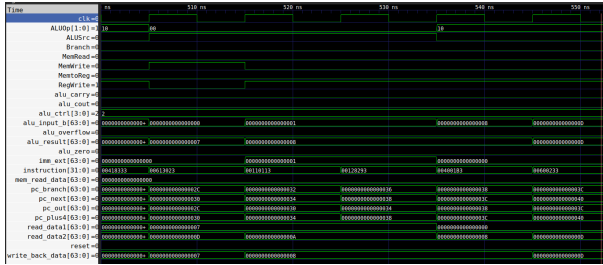


Fig. 7. Waveform at 500 ns

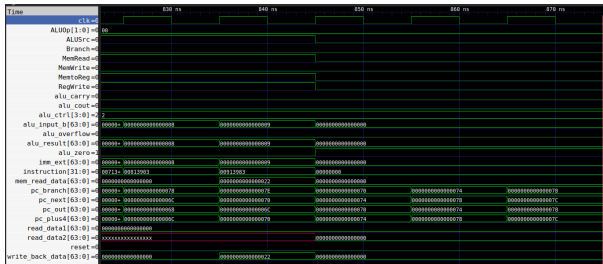


Fig. 8. Final waveform at program completion

X. CONCLUSION AND FUTURE WORK

This project successfully implemented a sequential single-cycle RISC-V processor using a modular datapath approach in Verilog HDL. All core components including instruction fetch, decode, execution, memory access, and write-back were designed and integrated correctly.

The processor was able to execute a meaningful subset of the RISC-V instruction set including arithmetic operations, memory access, and conditional branching. Simulation results verified the correctness of control signals, datapath flow, and state updates across clock cycles.

Although the sequential design simplifies control and verification, it limits performance due to long critical paths and low instruction throughput.

Future enhancements may include:

- Transition to a pipelined datapath for improved performance
- Hazard detection and forwarding mechanisms
- Support for additional RISC-V instructions
- Cache integration for faster memory access

This work establishes a strong architectural foundation for exploring advanced processor design concepts. The register file updates the destination register on the active clock edge.